Experiment – 10

Name-Leonhard Dsouza Div-SE-CompsA Batch-B Roll no-24

File Allocation Strategies	
Learning Objective:	Write a program to simulate File allocation strategies typically sequential, indexed and linked files
Learning Outcome:	Students will be able to implement file allocation strategies
Course Outcome:	**CSL403.5**
Program Outcome:	(PO 3) Design/ development of solutions: Breadth and uniqueness of engineering problems i.e. the extent to which problems are original and to which solutions have previously been identified or codified (PO 12) Life Long Learning
Bloom's Taxonomy Level:	Understand Analyze
Theory:	File allocation refers to the method used by an operating system to store and organize files on a storage device. The efficiency of file allocation strategies impacts storage utilization, retrieval speed, and fragmentation levels. The three primary file allocation strategies are Contiguous Allocation, Linked Allocation, and Indexed Allocation.

Algorithm :	**Algorithm for File Allocation Strategies**
	Step 1: Define Constants and Structures
	• Define MAX_FILES (Maximum number of files).
	• Define MAX_BLOCKS (Maximum blocks available).
	• Create a structure File containing:
	  o fileID (Unique identifier for each file).
	  o size (Size of the file in blocks).
	  o blocks[MAX_BLOCKS] (Array to store allocated blocks).
	Step 2: Input File Information
	1. Prompt the user to enter the number of files.
	2. Validate the input to ensure it does not exceed MAX_FILES or is not negative.
	3. For each file, input its size (in blocks).
	4. Validate the size to ensure it does not exceed MAX_BLOCKS or is not negative.
	Step 3: Implement Sequential Allocation
	1. Initialize currentBlock = 0.
	2. For each file:
	  o Allocate size blocks starting from currentBlock.
	  o Store the allocated blocks in blocks array.
	  o Increment currentBlock accordingly.
	3. Print allocated blocks for each file.
	Step 4: Implement Indexed Allocation
	1. For each file:
	  o Print the memory address of its index block (for simulation).
	  o Assign random blocks (within MAX_BLOCKS) to the file's blocks array.
	  o Print allocated blocks.

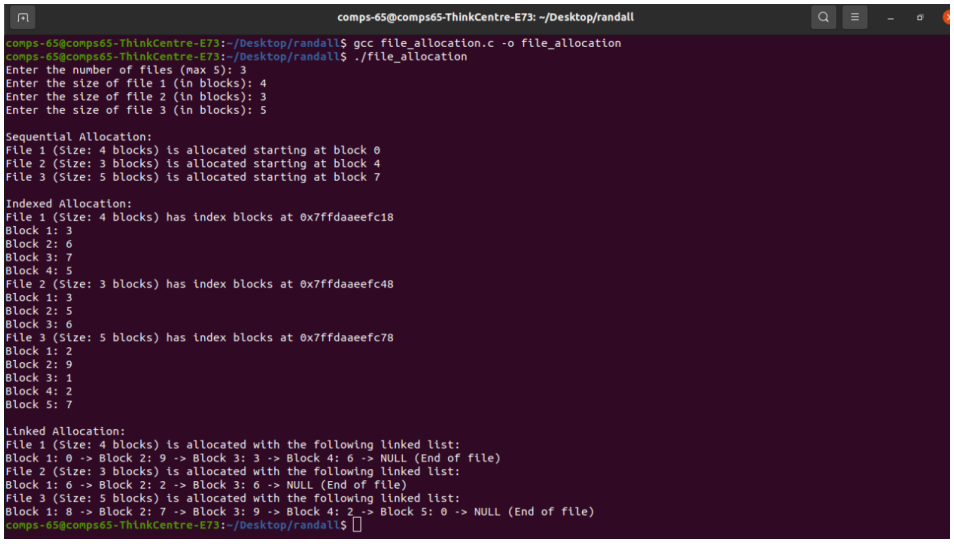
	Step 5: Implement Linked Allocation
	1. For each file:
	  - Assign the first block randomly within MAX_BLOCKS.
	  - Store each block's reference to the next allocated block.
	  - Print the linked list of allocated blocks.
	Step 6: Execution and Output
	1. Execute **Sequential, Indexed, and Linked Allocation** functions in sequence.
	2. Display allocated blocks for each strategy.
	3. End the program.

Data Set :	Assume Appropriate Data Set

Outcome :	#include <stdio.h>
	#include <stdlib.h>
	#include <string.h>
	
	#define MAX_FILES 5
	#define MAX_BLOCKS 10
	
	// Struct to simulate a file
	typedef struct {
	  int fileID;
	  int size; // Size of the file in blocks
	  int blocks[MAX_BLOCKS];
	} File;
	
	// Sequential File Allocation
	void sequentialAllocation(File *files, int numFiles) {
	  printf("\nSequential Allocation:\n");
	  int currentBlock = 0;
	
	  for (int i = 0; i < numFiles; i++) {
	    printf("File %d (Size: %d blocks) is allocated starting at block %d\n", files[i].fileID, files[i].size, currentBlock);
	    for (int j = 0; j < files[i].size; j++) {
	      files[i].blocks[j] = currentBlock++;
	    }
	  }
	}
	
	// Indexed File Allocation
	void indexedAllocation(File *files, int numFiles) {
	  printf("\nIndexed Allocation:\n");
	  for (int i = 0; i < numFiles; i++) {
	    printf("File %d (Size: %d blocks) has index blocks at %p\n", files[i].fileID, files[i].size, files[i].blocks);
	    for (int j = 0; j < files[i].size; j++) {
	      files[i].blocks[j] = rand() % MAX_BLOCKS; // Assigning random blocks (for simulation purposes)

```
            printf("Block %d: %d\n", j + 1, files[i].blocks[j]);
        }
    }
}

// Linked File Allocation
void linkedAllocation(File *files, int numFiles) {
    printf("\nLinked Allocation:\n");
    int prevBlock = -1;
    for (int i = 0; i < numFiles; i++) {
        printf("File %d (Size: %d blocks) is allocated with the following
linked list:\n", files[i].fileID, files[i].size);
        prevBlock = rand() % MAX_BLOCKS;
        for (int j = 0; j < files[i].size; j++) {
            files[i].blocks[j] = prevBlock;
            printf("Block %d: %d -> ", j + 1, files[i].blocks[j]);
            prevBlock = rand() % MAX_BLOCKS;
        }
        printf("NULL (End of file)\n");
    }
}

int main() {
    File files[MAX_FILES];
    int numFiles;

    // Input number of files and their sizes
    printf("Enter the number of files (max %d): ", MAX_FILES);
    scanf("%d", &numFiles);
    if (numFiles > MAX_FILES || numFiles <= 0) {
        printf("Invalid number of files! Exiting...\n");
        return 1;
    }

    for (int i = 0; i < numFiles; i++) {
        files[i].fileID = i + 1;
        printf("Enter the size of file %d (in blocks): ", files[i].fileID);
        scanf("%d", &files[i].size);
        if (files[i].size <= 0 || files[i].size > MAX_BLOCKS) {
            printf("Invalid file size! Exiting...\n");
            return 1;
        }
    }

    // Simulating each file allocation strategy
    sequentialAllocation(files, numFiles);
    indexedAllocation(files, numFiles);
    linkedAllocation(files, numFiles);
```

```
    return 0;
}
```



Conclusion :	The given program is successfully implemented
References:	Give References
Questions:	1. Explain file allocation methods in detail with proper diagram. File allocation refers to the method of storing files on a disk. The three main file allocation methods are: **a) Sequential Allocation** • In **sequential allocation**, files are stored in contiguous memory blocks. • The operating system keeps track of the starting block and the number of blocks allocated. **Diagram:** | File1 | File2 | File3 | Free | Free | File4 | **b) Linked Allocation** • Each file is stored as a linked list of blocks scattered across the disk. • Each block contains data and a pointer to the next block. **Diagram:** File1 -> Block 5 -> Block 9 -> Block 12 -> NULL File2 -> Block 3 -> Block 8 -> NULL **c) Indexed Allocation** • An **index block** is maintained that stores pointers to file blocks. • The index contains addresses of all blocks used by the file. **Diagram:** Index Block -> Block 3, Block 7, Block 15, Block 20 2. Compare append mode operation for sequential and random access file. **Feature** **Sequential Access File** **Random Access File**

	Definition	Data is read/written in a linear sequence.	Data can be read/written at any positio
	Appending	Always adds data at the end of the file.	Can append data anywhere in the file using file pointers.
	Performan ce	Faster for continuous writing.	Slower for appending in between due repositioning.
	Use Cases	Log files, transaction records.	Databases, multimedia files.